



Informe del Proceso ETL — Reto 3

Reto 3 — Diseño y Optimización de Flujos de Trabajo ETL Avanzados

Autor: Judit Giravent

Curso: Business Intelligence y Big Data (Nivel 5) | Odisea Data

Herramienta ETL: Apache NiFi 1.28.1

Fecha: Septiembre 2026

1. Introducción

Este proyecto implementa un **flujo ETL avanzado** con Apache NiFi capaz de integrar **3 fuentes de datos heterogéneas** (CSV, API REST y SQLite) en un único dataset final. El pipeline sigue las 7 fases del reto: selección de herramienta, recolección de datos, diseño del flujo, automatización, implementación, validación y documentación.

2. Selección de la herramienta ETL

Comparativa evaluada

Herramienta	Coste	Facilidad	UI Visual	Automatización	Decisión
Apache NiFi	✅ 100% Open Source	Media (curva de aprendizaje)	✅ Muy potente	✅ Nativa	Elegida
Airbyte	⚠️ Open Source con licencia ELv2	Alta	✅ Buena	✅	Descartada (requiere Docker)
Mage	❌ Sin plan gratuito permanente (\$100/mes)	Alta	✅ Buena	✅	Descartada (coste)
Hevodata	⚠️ Solo versión trial	Alta	✅ Buena	✅	Descartada (trial limitado)
CloudQuery	✅ Open Source	Media	❌ CLI, sin UI	✅	Descartada (sin UI visual)

Justificación: Apache NiFi es la única opción **100% gratuita y sin limitaciones**, con interfaz visual, soporte para múltiples fuentes y potentes capacidades de transformación y automatización.

3. Recolección de datos

Fuente 1 — CSV: Ventas Retail

- **Origen:** Dataset Online Retail (Kaggle)

- **Tipo:** Archivo CSV
- **Registros originales:** 541.909
- **Registros usados:** 10.000 (versión reducida `OnlineRetail_10k.csv`)
- **Campos:** `InvoiceNo`, `StockCode`, `Description`, `Quantity`, `InvoiceDate`, `UnitPrice`, `CustomerID`, `Country`

Fuente 2 — API REST: Precios de Criptomonedas

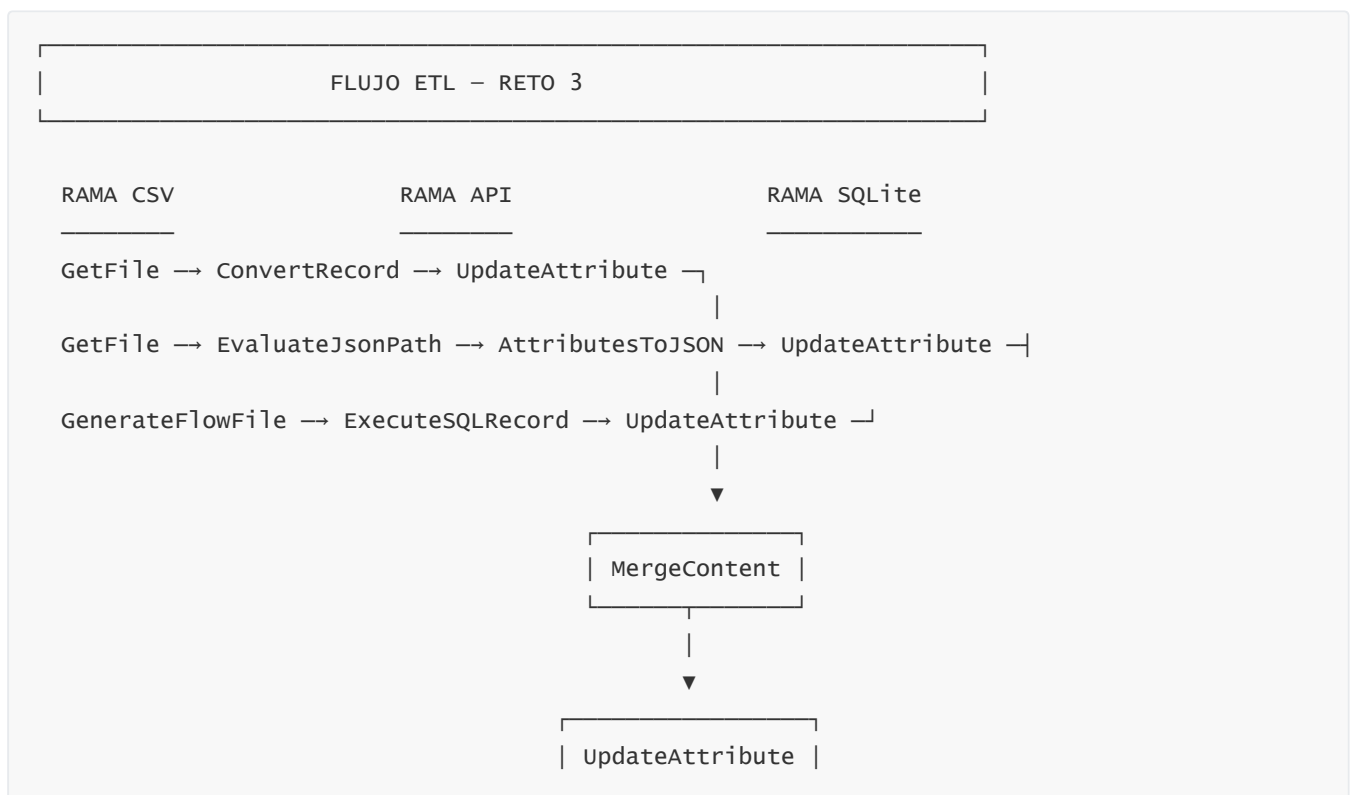
- **Origen:** CoinGecko API (<https://api.coingecko.com/api/v3/simple/price>)
- **Tipo:** JSON vía HTTPS
- **Datos extraídos:** Precios de Bitcoin, Ethereum, Cardano y Solana en USD y EUR
- **Nota:** Se usó un JSON estático descargado previamente con PowerShell por limitaciones de NiFi con el User-Agent requerido por CoinGecko (ver sección 7 de problemas encontrados)

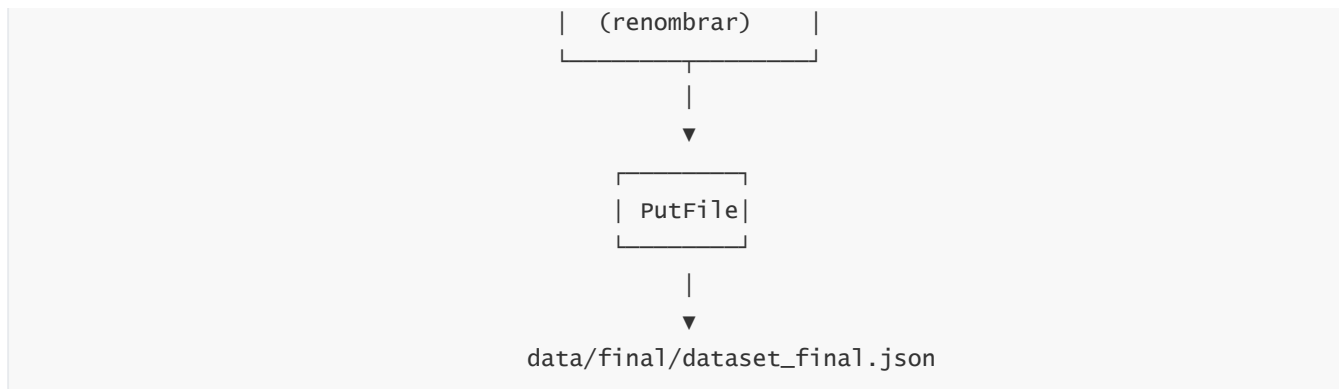
Fuente 3 — SQLite: Tabla de Clientes

- **Origen:** Base de datos `clientes.db` generada con Python
- **Tipo:** SQLite (SQL)
- **Registros:** 2.000 clientes
- **Campos:** `cliente_id`, `nombre`, `email`, `edad`, `genero`, `pais`, `ciudad`, `fecha_registro`, `tipo_cliente`, `ingresos_anuales`

4. Diseño del flujo ETL

Arquitectura general





Procesadores por rama

Rama CSV

Procesador	Función
<code>GetFile</code>	Lee <code>onlineRetail_10k.csv</code> desde <code>data/raw/csv/</code>
<code>ConvertRecord</code>	Convierte CSV a JSON con <code>CSVReader</code> + <code>JsonRecordSetWriter</code>
<code>UpdateAttribute</code>	Renombra a <code>ventas_YYYYMMDD_HHMMSS.json</code>
<code>PutFile</code>	Guarda en <code>data/processed/ventas/</code>

Rama API

Procesador	Función
<code>GetFile</code>	Lee el JSON de crypto descargado desde <code>data/raw/api/</code>
<code>EvaluateJsonPath</code>	Extrae los 6 valores (<code>btc_usd</code> , <code>btc_eur</code> , <code>eth_usd</code> , <code>eth_eur</code> , <code>ada_usd</code> , <code>sol_usd</code>)
<code>AttributesToJSON</code>	Convierte los atributos en un JSON limpio
<code>UpdateAttribute</code>	Renombra a <code>cripto_YYYYMMDD_HHMMSS.json</code>
<code>PutFile</code>	Guarda en <code>data/processed/api/</code>

Rama SQLite

Procesador	Función
<code>GenerateFlowFile</code>	Dispara la ejecución del SQL
<code>ExecuteSQLRecord</code>	Ejecuta <code>SELECT * FROM clientes</code> sobre <code>clientes.db</code>
<code>UpdateAttribute</code>	Renombra a <code>clientes_YYYYMMDD_HHMMSS.json</code>
<code>PutFile</code>	Guarda en <code>data/processed/clientes/</code>

Unión final

Procesador	Función
MergeContent	Une los 3 JSONs con Binary Concatenation
UpdateAttribute	Renombra a dataset_final.json
PutFile	Guarda en data/final/

5. Automatización

Programación de ejecuciones

Procesador	Frecuencia	Justificación
GetFile (CSV)	1 day	Los datos de ventas cambian raramente
GetFile (API)	1 day	Los precios de cripto cambian constantemente, pero usamos JSON estático
GenerateFlowFile (SQLite)	1 day	Los datos de clientes cambian raramente

Estrategia

El flujo está diseñado para ejecutarse **manualmente** durante el desarrollo y **diariamente en producción**. Todas las ramas convergen en el MergeContent, que espera a tener **3 FlowFiles** (uno por rama) para ejecutar la unión.

6. Validación de datos

Para cada rama se validó:

- 1. **Integridad:** todos los registros se han procesado sin pérdidas.
- 2. **Formato:** la conversión de CSV a JSON y SQL a JSON mantiene los tipos.
- 3. **Compleitud:** el JSON final contiene los 3 bloques (ventas, cripto, clientes).